Exact Evaluation of High-Order Mixed Derivatives by Rank-Optimal Directional Taylor Jets

Abstract

Repeated evaluation of a prescribed high-order mixed derivative can dominate the cost of neural methods for partial differential equations. We study exact complex directional formulas that express one order-p mixed partial as a weighted sum of order-p Taylor coefficients along straight lines. Universal exactness is equivalent to a Waring decomposition over $\mathbb{C}$ of the monomial associated with the target multiplicity pattern. The complex Waring rank therefore gives the minimum number of terms in this formula class, and a roots-of-unity construction provides an explicit rank-optimal schedule. When the derivative tensor is real, conjugate terms can be paired so that only one Taylor jet is evaluated per conjugation orbit. For the patterns $(2,2)$, $(4,2)$, and $(2,2,2)$, the complex ranks are 3, 5, and 9, whereas the paired implementation uses 2, 3, and 5 jet calls. Each directional coefficient is evaluated by normalized Taylor-mode propagation, and a custom vector–Jacobian product keeps the result differentiable with respect to model parameters. The numerical study uses nested automatic differentiation as the sole reference implementation. The network, residual, sampling points, initialization, optimizer, and precision are held fixed, so the derivative backend is the only changed component. The optimality result is restricted to complex straight-line directional formulas; algebraic term counts and jet-call counts are not interpreted as wall-clock speedups without controlled timing and memory measurements.

Keywords. High-order automatic differentiation; Taylor-mode differentiation; complex Waring decomposition; mixed partial derivatives; physics-informed neural networks.

MSC 2020. 65D25, 68T07, 14A25, 35J30.

1 Introduction

Neural methods for partial differential equations now include stochastic, variational, and residual formulations [13, 10, 12, 29, 18, 8]. In a physics-informed neural network (PINN), each training step evaluates the coordinate derivatives required by the differential operator and then differentiates the residual loss with respect to the model parameters [29, 11]. When the residual contains a prescribed fourth- or sixth-order mixed partial, successive applications of automatic differentiation construct nested derivative graphs. High-order equations make this burden particularly visible; one alternative is to reformulate the PDE as a lower-order mixed system rather than compute the original high-order operator directly [26].

Automatic differentiation and its forward- and reverse-mode constructions are standard tools in scientific computing and machine learning [14, 1, 28]. Taylor-mode automatic differentiation propagates a truncated univariate Taylor series through the computational graph [3, 31]. Recent work reduces derivative cost by exploiting coordinate structure, trace structure, general Taylor paths, joint propagation of operator coefficients, or residual-specific kernels [23, 6, 24, 30, 9, 4]. Encoding and representation choices can also change the cost and regularity of coordinate derivatives [16]. These approaches do not evaluate the same restricted algebraic object studied here. We use them to delimit the scope of the present result, but do not combine them in the numerical comparison.

The outcome of a PINN experiment also depends on the loss formulation, optimization dynamics, collocation set, and numerical protocol [32, 33, 21, 34, 19, 2, 15]. For this reason, the

experiments compare the proposed backend only with nested automatic differentiation and keep every other part of the model and training pipeline unchanged. The comparison is intended to isolate the derivative backend rather than rank unrelated PINN formulations.

Let $\partial_I^{\boldsymbol{\alpha}} u(x)$ be one prescribed order-p mixed partial. We consider identities

$$\partial_I^{\boldsymbol{\alpha}} u(x) = \sum_{r=1}^{R} \lambda_r \mathcal{T}_p[u](x; v_r), \qquad \lambda_r \in \mathbb{C}, \quad v_r \in V_{\mathbb{C}}, \tag{1}$$

where $\mathcal{T}_p[u](x; v_r)$ is the normalized order-p contraction of the derivative tensor along the complex direction v_r. The identity is required to hold for every admissible order-p tensor. We seek the smallest number R of complex straight-line directional Taylor coefficients. General Taylor paths with nonzero higher-order input coefficients, operator-level graph transformations, and residual-specific kernels lie outside this formula class.

The answer is algebraic. The target mixed partial is one coefficient of a homogeneous polynomial in the direction variables. Equation (1) is exact precisely when the corresponding monomial is represented as a weighted sum of p-th powers of complex linear forms. This is the homogeneous-polynomial form of a symmetric tensor-rank decomposition [7, 22]. The minimum value of R is therefore the complex Waring rank of that monomial. This rank is known in closed form [5], and a roots-of-unity decomposition yields an explicit minimal schedule.

The algebraic schedule is combined with normalized Taylor-mode propagation. Each complex direction produces one Taylor coefficient, and a custom reverse rule differentiates the result with respect to the model parameters. Complex directions are interpreted through the complexification of the real-coordinate derivative tensor; the construction does not require a general real function to be evaluated at arbitrary complex points. When the derivative tensor is real, the schedule is closed under complex conjugation. A conjugate pair can then be evaluated from one complex jet followed by a real-part operation. This reduces the number of jet calls without changing the complex Waring rank.

The contributions are as follows.

(i) We characterize universally exact complex formulas of the form (1) by power-sum decompositions of the target monomial. This identifies the minimum formula length with the complex Waring rank.

(ii) We convert the known complex monomial decomposition into an explicit roots-of-unity derivative schedule and derive a conjugate-orbit implementation for real derivative tensors, including its exact jet-call count.

(iii) We implement the schedule with normalized Taylor jets and a custom vector–Jacobian product. Numerical verification and performance tests use nested automatic differentiation as the only reference and change only the derivative backend.

Section 2 formulates the coefficient-extraction problem. Section 3 gives the complex power-sum characterization, the rank-optimal roots-of-unity schedule, and the conjugate-orbit reduction. Section 4 describes Taylor-jet evaluation and reverse differentiation. Section 5 specifies the controlled comparison with nested automatic differentiation, and Sections 6 and 7 discuss the scope and limitations of the method.

2 Problem Statement and Coefficient Extraction

2.1 Target derivative and active coordinates

Let $u : \mathbb{R}^d \to \mathbb{C}$ be p times continuously differentiable with respect to its real coordinates in a neighborhood of x; real-valued functions are included as a special case. A prescribed order-p

derivative involves only a subset of the coordinates. We denote the active coordinates by

$$I = (i_1, \dots, i_s)$$

and their positive multiplicities by

$$\boldsymbol{a} = (a_1, \dots, a_s), \qquad 1 \le a_1 \le \cdots \le a_s, \qquad p = \sum_{j=1}^{s} a_j.$$

The ordering is by multiplicity; ties may be resolved by the coordinate index. The target derivative is

$$\partial_I^{\boldsymbol{a}} u(x) := \partial_{i_1}^{a_1} \cdots \partial_{i_s}^{a_s} u(x). \tag{2}$$

We use

$$\boldsymbol{a}! := \prod_{j=1}^{s} a_j!, \qquad \boldsymbol{z}^{\boldsymbol{a}} := \prod_{j=1}^{s} z_j^{a_j}.$$

Only the active coordinate space

$$V := \operatorname{span}\{e_{i_1}, \dots, e_{i_s}\} \subset \mathbb{R}^d \tag{3}$$

is relevant. Its complexification is $V_{\mathbb{C}} := V \otimes_{\mathbb{R}} \mathbb{C}$.

For $v \in V_{\mathbb{C}}$, define the normalized order-p directional Taylor coefficient by

$$\mathcal{T}_p[u](x; v) := \frac{1}{p!} D^p u(x)_{\mathbb{C}}[v, \dots, v], \tag{4}$$

where $D^p u(x)_{\mathbb{C}}$ is the complex-multilinear extension of the real-coordinate derivative tensor. When $v \in V$, this reduces to

$$\mathcal{T}_p[u](x; v) = \frac{1}{p!} \frac{d^p}{d\tau^p} u(x + \tau v)\Big|_{\tau=0}.$$

Thus, complex directions are tensor contractions; a general real function need not be defined on a complex neighborhood of x.

2.2 Generating polynomial

For $\boldsymbol{z} = (z_1, \dots, z_s) \in \mathbb{C}^s$, define

$$G_x(\boldsymbol{z}) := \mathcal{T}_p[u]\left(x; \sum_{j=1}^{s} z_j e_{i_j}\right). \tag{5}$$

The polynomial $G_x \in \mathbb{C}[\boldsymbol{z}]_p$ is homogeneous of degree p.

Lemma 2.1 (Generating polynomial)**.** *For every $u \in C^p$ near x,*

$$G_x(\boldsymbol{z}) = \sum_{|\boldsymbol{b}|=p} \frac{\partial_I^{\boldsymbol{b}} u(x)}{\boldsymbol{b}!} \boldsymbol{z}^{\boldsymbol{b}}, \tag{6}$$

where $\boldsymbol{b} = (b_1, \dots, b_s) \in \mathbb{N}_0^s$, $|\boldsymbol{b}| = \sum_j b_j$, and $\partial_I^{\boldsymbol{b}} = \partial_{i_1}^{b_1} \cdots \partial_{i_s}^{b_s}$.

Proof. Apply the multinomial theorem to the complexification of the symmetric multilinear form $D^p u(x)$:

$$D^p u(x)_{\mathbb{C}}\left[\sum_j z_j e_{i_j}, \dots, \sum_j z_j e_{i_j}\right] = \sum_{|\boldsymbol{b}|=p} \frac{p!}{\boldsymbol{b}!} \boldsymbol{z}^{\boldsymbol{b}} \partial_I^{\boldsymbol{b}} u(x).$$

Division by $p!$ gives (6). □

It follows that

$$\partial_I^{\boldsymbol{a}} u(x) = \boldsymbol{a}! \, [\boldsymbol{z}^{\boldsymbol{a}}] G_x(\boldsymbol{z}), \tag{7}$$

where $[\boldsymbol{z}^{\boldsymbol{a}}]$ denotes coefficient extraction.

2.3 Complex straight-line directional formulas

Definition 2.2 (Complex straight-line directional formula)**.** A complex directional formula of length R for $\partial_I^{\boldsymbol{a}}$ is a collection

$$(\lambda_r, v_r)_{r=1}^R \subset \mathbb{C} \times V_{\mathbb{C}}$$

such that

$$\partial_I^{\boldsymbol{a}} u(x) = \sum_{r=1}^R \lambda_r \, \mathcal{T}_p[u](x; v_r) \tag{8}$$

for every complex symmetric order-p tensor on the active coordinate space.

Universal exactness reduces (8) to a finite-dimensional coefficient identity. The next section identifies that identity and its minimum complex length.

3 Complex Rank-Optimal Directional Formulas

3.1 Power-sum characterization

For $v = \sum_{j=1}^s v_j e_{i_j} \in V_{\mathbb{C}}$, write

$$v \cdot \boldsymbol{z} := \sum_{j=1}^s v_j z_j.$$

Definition 3.1 (Complex Waring rank)**.** For a homogeneous polynomial $F \in \mathbb{C}[\boldsymbol{z}]_p$, its complex Waring rank is

$$R_{\mathbb{C}}(F) := \min \left\{ R : F = \sum_{r=1}^R \lambda_r L_r(\boldsymbol{z})^p, \quad \lambda_r \in \mathbb{C}, \quad L_r \in \mathbb{C}[\boldsymbol{z}]_1 \right\}.$$

Theorem 3.2 (Power-sum characterization)**.** *For $(\lambda_r, v_r)_{r=1}^R \subset \mathbb{C} \times V_{\mathbb{C}}$, the following statements are equivalent:*

(i) *The directional identity (8) holds for every complex symmetric order-p tensor on the active coordinate space.*

(ii) *The polynomial identity*

$$p!\, \boldsymbol{z}^{\boldsymbol{a}} = \sum_{r=1}^R \lambda_r (v_r \cdot \boldsymbol{z})^p \quad \text{in } \mathbb{C}[\boldsymbol{z}]_p \tag{9}$$

holds.

Proof. By Lemma 2.1,

$$\mathcal{T}_p[u](x; v_r) = \sum_{|\boldsymbol{b}|=p} \frac{\partial_I^{\boldsymbol{b}} u(x)}{\boldsymbol{b}!} v_r^{\boldsymbol{b}}.$$

Because the active complex order-p tensor is arbitrary, (8) is equivalent to

$$\sum_{r=1}^R \lambda_r v_r^{\boldsymbol{b}} = \boldsymbol{a}!\, \delta_{\boldsymbol{a}\boldsymbol{b}}, \qquad |\boldsymbol{b}| = p. \tag{10}$$

The coefficient of $\boldsymbol{z}^{\boldsymbol{b}}$ in the right-hand side of (9) is

$$\binom{p}{\boldsymbol{b}} \sum_{r=1}^R \lambda_r v_r^{\boldsymbol{b}}.$$

Coefficient comparison, together with $p!/\binom{p}{\boldsymbol{b}} = \boldsymbol{b}!$, gives exactly (10). □

Corollary 3.3. *The minimum length of a universally exact complex directional formula for $\partial_I^{\boldsymbol{a}}$ is $R_{\mathbb{C}}(\boldsymbol{z}^{\boldsymbol{a}})$.*

Remark 3.4 (Scope of optimality). Corollary 3.3 optimizes complex formulas that use order-p coefficients of straight-line directions. It does not optimize over general Taylor paths $x + \tau v_1 + \tau^2 v_2 + \cdots$, methods that propagate several operator coefficients jointly, or residual-specific symbolic kernels. Those methods use a different computational class.

3.2 Explicit complex schedule

The complex Waring rank of a monomial is known exactly.

Theorem 3.5 (Carlini–Catalisano–Geramita [5])**.** *Let $1 \le a_1 \le \cdots \le a_s$. Then*

$$R_{\mathbb{C}}(\boldsymbol{z}^{\boldsymbol{a}}) = \prod_{j=2}^{s}(a_j+1) = \frac{\prod_{j=1}^{s}(a_j+1)}{a_1+1}. \tag{11}$$

For $s = 1$, the empty product equals 1.

Theorem 3.5 supplies the lower bound for the number of complex straight-line jets. The next construction attains it and provides a schedule that can be used directly in an implementation.

Lemma 3.6 (Roots-of-unity filter)**.** *For $m \in \mathbb{N}$ and $k \in \mathbb{Z}$,*

$$\sum_{\zeta^m=1} \zeta^k = m\,\mathbf{1}_{\{m|k\}}. \tag{12}$$

Proof. If $m \mid k$, every term equals 1. Otherwise, the sum is a nontrivial finite geometric series and vanishes. □

Theorem 3.7 (Roots-of-unity schedule)**.** *For $j = 2, \dots, s$, let $m_j = a_j + 1$, and define*

$$\Omega := \left\{\boldsymbol{\zeta} = (\zeta_2, \dots, \zeta_s) : \zeta_j^{m_j} = 1\right\}, \qquad R := |\Omega| = \prod_{j=2}^{s} m_j.$$

For each $\boldsymbol{\zeta} \in \Omega$, set

$$v_{\boldsymbol{\zeta}} = e_{i_1} + \sum_{j=2}^{s} \zeta_j e_{i_j}, \qquad \lambda_{\boldsymbol{\zeta}} = \frac{\boldsymbol{a}!}{R} \prod_{j=2}^{s} \zeta_j. \tag{13}$$

Then

$$\partial_I^{\boldsymbol{a}} u(x) = \sum_{\boldsymbol{\zeta}\in\Omega} \lambda_{\boldsymbol{\zeta}}\, \mathcal{T}_p[u](x; v_{\boldsymbol{\zeta}}). \tag{14}$$

The formula has length $R = R_{\mathbb{C}}(\boldsymbol{z}^{\boldsymbol{a}})$ and is therefore minimal among complex straight-line directional formulas.

Proof. By Theorem 3.2, it is enough to prove

$$p!\, \boldsymbol{z}^{\boldsymbol{a}} = \sum_{\boldsymbol{\zeta}\in\Omega} \lambda_{\boldsymbol{\zeta}} (v_{\boldsymbol{\zeta}} \cdot \boldsymbol{z})^p.$$

For $|\boldsymbol{b}| = p$, the coefficient of $\boldsymbol{z}^{\boldsymbol{b}}$ on the right-hand side is

$$\frac{\boldsymbol{a}!}{R}\binom{p}{\boldsymbol{b}} \prod_{j=2}^{s} \left(\sum_{\zeta_j^{m_j}=1} \zeta_j^{b_j+1} \right).$$

By Lemma 3.6, this coefficient vanishes unless $b_j \equiv a_j \pmod{a_j+1}$ for every $j \geq 2$. Hence

$$b_j = a_j + t_j(a_j+1), \qquad t_j \in \mathbb{N}_0.$$

If some $t_j > 0$, then

$$b_1 = a_1 - \sum_{j=2}^{s} t_j(a_j+1) < 0,$$

which is impossible because $a_j \geq a_1$. Thus $\boldsymbol{b} = \boldsymbol{a}$ is the only surviving index. For this index, the root sums cancel R, and the coefficient is

$$\boldsymbol{a}!\binom{p}{\boldsymbol{a}} = p!.$$

This proves the polynomial identity. Minimality follows from Theorem 3.5. □

Remark 3.8 (Meaning of complex directions). For a general real C^p function, equation (14) is an identity in the complexification of $D^p u(x)$; it is not an instruction to evaluate u at arbitrary complex points. A direct complex-jet implementation realizes the same tensor contractions when all computational primitives support the required complex derivatives.

3.3 Conjugate-orbit evaluation for real derivative tensors

The roots-of-unity set Ω is invariant under componentwise complex conjugation. Moreover,

$$v_{\overline{\zeta}} = \overline{v_\zeta}, \qquad \lambda_{\overline{\zeta}} = \overline{\lambda_\zeta}.$$

This symmetry reduces the number of jet evaluations when the derivative tensor is real.

Proposition 3.9 (Conjugate-orbit formula)**.** *Assume that $D^p u(x)$ is real. Let*

$$\Omega_0 := \{\zeta \in \Omega : \overline{\zeta} = \zeta\},$$

and choose one representative from each two-element conjugate orbit to form Ω_+. Then

$$\partial_I^{\boldsymbol{a}} u(x) = \sum_{\zeta \in \Omega_0} \lambda_\zeta \, \mathcal{T}_p[u](x; v_\zeta) + 2\,\mathrm{Re} \sum_{\zeta \in \Omega_+} \lambda_\zeta \, \mathcal{T}_p[u](x; v_\zeta). \tag{15}$$

The number of Taylor-jet evaluations is

$$R_{\mathrm{orb}} = \frac{R+F}{2}, \qquad F = \prod_{j=2}^{s} \gcd(2, a_j+1), \tag{16}$$

with the empty product interpreted as one.

Proof. Real multilinearity gives $\mathcal{T}_p[u](x;\overline{v}) = \overline{\mathcal{T}_p[u](x;v)}$. Hence the two terms indexed by ζ and $\overline{\zeta}$ sum to $2\,\mathrm{Re}(\lambda_\zeta \mathcal{T}_p[u](x;v_\zeta))$. A root of unity is fixed by conjugation exactly when it is real. The equation $\zeta^{a_j+1} = 1$ has $\gcd(2, a_j+1)$ real solutions, so $|\Omega_0| = F$. The remaining $R-F$ elements form two-element orbits, which gives (16). □

Remark 3.10. Equation (15) is a real-linear implementation identity for real derivative tensors. It does not give a shorter complex Waring decomposition and therefore does not alter Theorem 3.5. A complex jet call is also not assumed to have the same cost as a real jet call.

pattern	$R_{\mathbb{C}}$	R_{orb}
(p)	1	1
$(2,1)$	3	2
$(1,1,1)$	4	4
$(3,1)$	4	3
$(2,2)$	3	2
$(2,1,1)$	6	4
(1^4)	8	8
$(4,2)$	5	3
$(2,2,2)$	9	5
(1^6)	32	32

Table 1: Complex Waring ranks and conjugate-orbit jet-call counts for representative multiplicity patterns. The orbit count applies only when the derivative tensor is real. A complex jet call is not assumed to have the same cost as a real automatic-differentiation sweep.

3.4 Complex ranks and conjugate-orbit call counts

Table 1 reports two distinct quantities. The complex Waring rank $R_{\mathbb{C}}$ is the minimum number of terms in a complex power-sum decomposition. For a real derivative tensor, R_{orb} is the number of Taylor jets evaluated after conjugate pairing. The latter is an implementation count and does not change the complex rank.

3.5 Examples

Example 3.11 (Second-order cross derivative)**.** For $\boldsymbol{a} = (1,1)$, the schedule has two directions:

$$\partial_{i_1}\partial_{i_2}u(x) = \frac{1}{2}\mathcal{T}_p[u](x; e_{i_1} + e_{i_2}) - \frac{1}{2}\mathcal{T}_p[u](x; e_{i_1} - e_{i_2}).$$

The associated polynomial identity is

$$2z_1z_2 = \frac{1}{2}(z_1 + z_2)^2 - \frac{1}{2}(z_1 - z_2)^2.$$

Example 3.12 (Sixth-order pattern $(2,2,2)$)**.** Let

$$\partial_I^{\boldsymbol{a}} u = \partial_{i_1}^2\partial_{i_2}^2\partial_{i_3}^2 u, \qquad \boldsymbol{a} = (2,2,2).$$

With $\omega = e^{2\pi \mathrm{i}/3}$ and $U_3 = \{1, \omega, \omega^2\}$, Theorem 3.7 gives nine directions

$$v_{\zeta_2,\zeta_3} = e_{i_1} + \zeta_2 e_{i_2} + \zeta_3 e_{i_3}, \qquad (\zeta_2, \zeta_3) \in U_3^2,$$

and weights

$$\lambda_{\zeta_2,\zeta_3} = \frac{8}{9}\zeta_2\zeta_3.$$

Therefore,

$$\partial_{i_1}^2\partial_{i_2}^2\partial_{i_3}^2 u(x) = \frac{8}{9}\sum_{\zeta_2,\zeta_3 \in U_3} \zeta_2\zeta_3\, \mathcal{T}_p[u]\left(x; e_{i_1} + \zeta_2 e_{i_2} + \zeta_3 e_{i_3}\right). \tag{17}$$

The complex rank is 9. The unique fixed conjugation index is $(\zeta_2, \zeta_3) = (1,1)$; the other eight indices form four conjugate pairs. Equation (15) therefore evaluates the same real derivative with five jet calls.

4 Taylor-Jet Evaluation and Differentiation

The previous section determines a set of directions and weights. This section describes the evaluation of each $\mathcal{T}_p[u](x;v)$, the reverse derivative with respect to network parameters, and the computational quantities used in the backend comparison.

4.1 Normalized Taylor jets

For a fixed direction v, consider the path

$$x(\tau) = x + \tau v.$$

Each intermediate quantity is represented by a normalized truncated Taylor series

$$y(\tau) = \sum_{k=0}^{p} y_k \tau^k + O(\tau^{p+1}), \qquad y_k = \frac{1}{k!}\frac{d^k y}{d\tau^k}(0).$$

The output coefficient y_p is $\mathcal{T}_p[u](x;v)$.

For an affine map $y = Wx + b$,

$$y_0 = Wx_0 + b, \qquad y_k = Wx_k, \quad k = 1, \dots, p. \tag{18}$$

For an entrywise activation $y = \phi(x)$, write

$$q(\tau) = \phi'(x(\tau)) = \sum_{k=0}^{p} q_k \tau^k.$$

The identity $y' = qx'$ gives

$$y_k = \frac{1}{k}\sum_{j=0}^{k-1}(k-j) q_j x_{k-j}, \qquad k = 1, \dots, p. \tag{19}$$

The coefficients q_j are obtained from the activation-specific recurrence.

For $y = \tanh(x)$, $q = 1 - y^2$, and

$$q_0 = 1 - y_0^2, \qquad q_k = -\sum_{j=0}^{k} y_j y_{k-j}, \quad k \geq 1. \tag{20}$$

For $y = \sinh(x)$, introduce $c = \cosh(x)$. Then

$$y_k = \frac{1}{k}\sum_{j=0}^{k-1}(k-j) c_j x_{k-j}, \tag{21}$$

$$c_k = \frac{1}{k}\sum_{j=0}^{k-1}(k-j) y_j x_{k-j}, \qquad k = 1, \dots, p. \tag{22}$$

These recurrences are algebraic and apply to real or complex coefficient tensors. The functions sinh and cosh are entire. The function tanh is meromorphic; its local recurrence is valid away from its poles, but it does not define a globally entire complex network.

4.2 Reverse differentiation through a jet

Differentiating through every scalar operation in the recurrence creates an unnecessarily large reverse graph. We instead use the linearized identity $\delta y = q\,\delta x$ in the truncated-series algebra. Let $\bar{y}_k$ be the cotangent of y_k. Under the complex cotangent convention used by our implementation, the activation pullback is

$$\bar{x}_m = \sum_{j=0}^{p-m} \overline{q_j}\,\bar{y}_{m+j}, \qquad m = 0, \dots, p. \tag{23}$$

The conjugation convention is framework dependent. The implementation is therefore checked by both finite-difference directional tests and complex inner-product tests, rather than by value comparisons alone. Equation (23) stores $p+1$ coefficient tensors per retained activation and avoids reverse differentiation through the individual recurrence steps.

4.3 Reference and proposed backends

The numerical study compares two derivative backends for a fixed target $\partial_I^{\boldsymbol{a}} u(x)$.

Nested automatic differentiation. The reference implementation uses PyTorch reverse-mode automatic differentiation [27]. It differentiates successively along an ordered list of p coordinate indices and retains the graph so that the residual remains differentiable with respect to model parameters.

Proposed Waring–Taylor backend. The proposed backend uses the rank-optimal roots-of-unity schedule in (14). For a general complex derivative tensor, it evaluates all

$$R_{\mathbb{C}}(\boldsymbol{z}^{\boldsymbol{a}}) = \prod_{j=2}^{s} (a_j + 1)$$

terms. When the derivative tensor is real, it uses the conjugate-orbit formula (15) and evaluates one representative per orbit. Nonreal directions are propagated by the complex Taylor-jet rules, and the weighted result remains differentiable through the custom reverse rule.

Both backends return the same target derivative in exact arithmetic. The comparison changes only how that derivative is evaluated; it does not change the model architecture, residual, sampling points, initialization, optimizer, or training schedule.

4.4 End-to-end proposed schedule

Algorithm 1 summarizes the full complex schedule.

Theorem 3.7 establishes algebraic exactness. For a real derivative tensor, the loop is taken over $\Omega_0 \cup \Omega_+$; fixed indices contribute once, and nonfixed representatives contribute through twice the real part as in (15). Numerical error is introduced only by finite-precision jet propagation and weighted summation.

4.5 Cost model and stability diagnostics

Let C_{lin} denote the cost of one forward pass through all affine maps for one coefficient tensor, and let N_{act} be the number of scalar activation entries processed in one network evaluation. With the recurrences written above, one straight-line jet has the value-evaluation cost

$$O\big(pC_{\mathrm{lin}} + p^2 N_{\mathrm{act}}\big)\,. \tag{24}$$

Algorithm 1 Proposed complex Waring–Taylor backend

Require: model u; point $x \in \mathbb{R}^d$; active coordinates $I = (i_1, \dots, i_s)$; multiplicities $1 \le a_1 \le \cdots \le a_s$

Ensure: $\partial_I^{\boldsymbol{a}} u(x)$

$p \leftarrow \sum_{j=1}^{s} a_j$
$m_j \leftarrow a_j + 1$ for $j = 2, \dots, s$
$\Omega \leftarrow \{(\zeta_2, \dots, \zeta_s) : \zeta_j^{m_j} = 1\}$
$R \leftarrow |\Omega|$, result $\leftarrow 0$
for all $\boldsymbol{\zeta} \in \Omega$ **do**
  $v \leftarrow e_{i_1} + \sum_{j=2}^{s} \zeta_j e_{i_j}$
  $\lambda \leftarrow (\boldsymbol{a}!/R) \prod_{j=2}^{s} \zeta_j$
  initialize $(x_0, x_1, \dots, x_p) \leftarrow (x, v, 0, \dots, 0)$
  propagate the jet through u using (18)–(22)
  result $\leftarrow$ result $+ \lambda y_p$
end for
return result

Let R_{call} denote the number of evaluated jets. It equals $R_{\mathbb{C}}$ for the full complex schedule and R_{orb} for the conjugate-orbit implementation on a real derivative tensor. Sequential evaluation multiplies the single-jet cost by R_{call} while keeping storage proportional to p coefficient tensors per retained activation. Fully batched evaluation exposes parallelism but increases coefficient storage proportionally to R_{call}. The constants differ between real and complex arithmetic and between value-only and parameter-gradient evaluations. Equation (24) is therefore a cost model, not a runtime result.

For each weighted sum, we report the cancellation indicator

$$\kappa_{\mathrm{sum}} := \frac{\sum_{r=1}^{R} |\lambda_r y_{p,r}|}{\max\left(\left|\sum_{r=1}^{R} \lambda_r y_{p,r}\right|, \varepsilon_{\mathrm{mach}}\right)}. \tag{25}$$

Large values identify cases in which rounding error can be amplified by cancellation. For real-valued targets evaluated with the full complex schedule, the normalized imaginary residual

$$\eta_{\mathrm{im}} := \frac{|\operatorname{Im}(\mathrm{result})|}{\max(|\operatorname{Re}(\mathrm{result})|, \varepsilon_{\mathrm{mach}})} \tag{26}$$

provides an additional implementation and stability diagnostic.

5 Numerical Evaluation

The numerical study compares only nested automatic differentiation and the proposed Waring–Taylor backend. This two-backend design is deliberate: it isolates the effect of replacing the derivative calculation and avoids mixing that effect with changes in the model architecture, operator representation, or training procedure. The reporting protocol follows the emphasis on fixed problem definitions, complete hyperparameters, and machine-readable benchmark records used in scientific-learning software and benchmark studies [11, 15].

5.1 Verification setup

The executable suite uses double precision for real tests and complex128 when complex propagation is required. It covers pure, repeated-index, and square-free patterns through order six, including $(2, 2)$, $(4, 2)$, $(2, 2, 2)$, and (1^6). For every test, nested automatic differentiation

produces the reference derivative and parameter gradient. The proposed backend uses the full complex schedule for a complex derivative tensor and the paired conjugate-orbit implementation for a real derivative tensor. Every output is an ordinary tensor, after which the residual is differentiated with respect to the same model parameters.

For a reference derivative d_{AD}, the proposed derivative d_{WT}, and their parameter gradients, we report

$$E_d = \frac{|d_{\mathrm{WT}} - d_{\mathrm{AD}}|}{\max(|d_{\mathrm{AD}}|, \varepsilon_{\mathrm{mach}})}, \qquad E_g = \frac{\|g_{\mathrm{WT}} - g_{\mathrm{AD}}\|_2}{\max(\|g_{\mathrm{AD}}\|_2, \varepsilon_{\mathrm{mach}})}. \tag{27}$$

Complex pullbacks are checked independently by an adjoint inner-product identity. The cancellation indicator (25) and, for the full complex schedule applied to a real target, the imaginary residual (26) are recorded as stability diagnostics.

5.2 Controlled computational comparison

The timing study uses the same model instance, parameter dtype, batch, target multi-index, and residual for both backends. Compilation and warm-up are reported separately from steady-state execution. The measured workloads are (i) derivative-value evaluation and (ii) a complete residual loss followed by parameter back-propagation. Each result reports synchronized median wall time, interquartile range, and peak allocated and reserved device memory. Sequential and batched implementations of the proposed schedule are reported separately. No speed or memory conclusion is drawn from the algebraic term count alone.

5.3 Controlled PDE training comparison

PDE experiments use one fixed network architecture for each problem. Paired runs start from identical parameters and use the same collocation points, minibatch order, residual weights, optimizer, learning-rate schedule, stopping rule, and evaluation grid. Because the residual-point distribution can itself change PINN accuracy, each pair reuses exactly the same sampled points [34]. The paired runs use AdamW, the decoupled-weight-decay variant of Adam, with identical hyperparameters for both derivative backends [20, 25]. No optimizer switch is made within a timed run. The only difference is whether the prescribed high-order derivative is evaluated by nested automatic differentiation or by the proposed Waring–Taylor backend. We report solution error as a function of both iteration count and wall time, together with total training time and peak memory. Existing experiments that change the architecture, activation, or representation are excluded from this comparison because they do not isolate the derivative backend.

6 Discussion and Limitations

6.1 Formula class and scope of optimality

The main result concerns one prescribed monomial derivative and one complex formula class: weighted sums of order-p Taylor coefficients along straight-line directions. Within this class, the minimum number of terms is the complex Waring rank. The result is independent of the network architecture. It does not provide a lower bound for general Taylor paths with higher-order input coefficients, methods that propagate several operator terms jointly, or residual-specific derivative kernels. The paper therefore makes no claim of numerical superiority over those broader approaches.

This scope also determines the experimental design. Nested automatic differentiation is used as the sole reference because it evaluates the same target derivative without changing the operator or model. This choice is also consistent with recent analyses that treat automatic

differentiation as a central component of neural PDE training rather than as an interchangeable finite-difference approximation [6]. The resulting comparison answers a specific question: what changes when the derivative backend is replaced by the proposed complex Waring–Taylor construction?

6.2 Conjugate pairing and call counts

The rank-optimal complex decomposition contains $R_{\mathbb{C}}$ powers. For a real derivative tensor, Equation (15) evaluates this same complex decomposition with one jet per conjugation orbit. This reduction uses conjugate symmetry and real-linear postprocessing; it is not a different Waring decomposition. A complex-valued target generally requires the full schedule. Moreover, one complex jet call need not have the same cost as one automatic-differentiation sweep, so R_{orb} is not a speedup factor.

6.3 Numerical stability and computational cost

Complex roots of unity lead to oscillatory weighted sums. Finite-precision error can therefore be amplified even when the formula is exact. The cancellation indicator κ_{sum}, the imaginary residual of the full schedule, and direct derivative and adjoint checks expose different failure modes. The paired formula removes the final imaginary cancellation for real targets but does not remove cancellation within each real-part contribution.

Runtime depends on more than the number of directions. Complex kernels have different throughput and storage requirements from real kernels. Batching reduces launch overhead but increases activation storage; sequential evaluation reduces memory but exposes less parallelism. Parameter back-propagation also includes the custom reverse rule. Consequently, efficacy, timing, and memory claims require the controlled two-backend measurements specified in Section 5.

6.4 Controlled PDE interpretation

The PDE comparison changes only the derivative backend. This removes the architecture and representation confounders that arise when different network classes are compared, but it also narrows the conclusion. Such controls are important because PINN training can be strongly affected by gradient imbalance, loss conditioning, and optimization pathologies [32, 33, 21]. A lower error at a fixed iteration count would indicate a numerical difference between derivative implementations; a lower error at a fixed wall time would additionally reflect their computational cost. Neither result would establish superiority over methods that reformulate the operator or use a different derivative class.

6.5 Reproducibility

The custom forward and reverse rules make executable validation essential. Each numerical result is tied to a versioned bundle containing the source commit, random seed, hardware, software environment, precision, architecture, derivative pattern, sampling protocol, optimizer settings, and evaluation procedure. Tables and figures are generated directly from these bundles rather than transcribed manually, following the reproducibility emphasis of modern PINN software and benchmark suites [11, 15].

7 Conclusion

We have characterized universally exact complex straight-line directional formulas for a prescribed high-order mixed derivative. The characterization is a power-sum identity for the associated monomial, so the minimum number of terms is its complex Waring rank. Combining

the known monomial-rank formula with a roots-of-unity decomposition gives an explicit rank-optimal complex schedule.

For real derivative tensors, conjugate terms can be evaluated by one jet per conjugation orbit. This reduces the jet-call counts for $(2,2)$, $(4,2)$, and $(2,2,2)$ from the complex ranks 3, 5, and 9 to 2, 3, and 5, respectively. The paired formula evaluates the same complex decomposition and does not change its rank.

Normalized Taylor propagation evaluates each directional coefficient, and a custom vector–Jacobian product preserves differentiation with respect to model parameters. The numerical study compares this backend only with nested automatic differentiation while holding the model, residual, data, and training procedure fixed. This design isolates the derivative backend. Practical speed and memory advantages remain empirical claims and must be supported by the controlled end-to-end measurements specified in the paper.

Data and code availability

The implementation and test suite are maintained in the project repository `https://github.com/freezeng123456/apolarity`. The final submission will identify the exact result-generating commit and a versioned archival release containing the machine-readable benchmark records.

A Apolar Interpretation of the Rank Formula

For completeness, we recall the algebraic object underlying Theorem 3.5. Let $F = \boldsymbol{z}^{\boldsymbol{a}} \in \mathbb{C}[\boldsymbol{z}]_p$, and let $\mathbb{C}[\boldsymbol{\partial}]$ act on $\mathbb{C}[\boldsymbol{z}]$ by differentiation. The apolar ideal of F is

$$F^{\perp} = \left(\partial_1^{a_1+1}, \dots, \partial_s^{a_s+1}\right). \tag{28}$$

It is a complete intersection. The apolarity lemma identifies a Waring decomposition of F with a reduced finite set of projective points whose vanishing ideal is contained in $F^{\perp}$. The Hilbert function of $\mathbb{C}[\boldsymbol{\partial}]/F^{\perp}$ gives the lower bound

$$R_{\mathbb{C}}(F) \geq \prod_{j=2}^{s} (a_j + 1),$$

and the roots-of-unity construction in Theorem 3.7 attains this bound. Detailed treatments are given in [17, 5, 22].

References

[1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, *Automatic differentiation in machine learning: a survey*, J. Mach. Learn. Res. **18**(153) (2018), 1–43.

[2] S. Berrone, C. Canuto, M. Pintore, *Variational physics informed neural networks: the role of quadratures and test functions*, J. Sci. Comput. **92** (2022), 100.

[3] J. Bettencourt, M. J. Johnson, D. Duvenaud, *Taylor-mode automatic differentiation for higher-order derivatives in JAX*, NeurIPS Workshop on Program Transformations for Machine Learning (2019).

[4] W. Cao, Z. Lu, W. Zhang, *Verified residual-specific explicit derivative kernels for physics-informed learning and discretized PDE adjoints*, arXiv:2606.29702 (2026).

[5] E. Carlini, M. V. Catalisano, A. V. Geramita, *The solution to the Waring problem for monomials and the sum of coprime monomials*, J. Algebra **370** (2012), 5–14.

[6] C. Chen, Y. Yang, Y. Xiang, W. Hao, *Automatic differentiation is essential in training neural networks for solving differential equations*, J. Sci. Comput. **104** (2025), 54.

[7] P. Comon, G. Golub, L.-H. Lim, B. Mourrain, *Symmetric tensors and symmetric tensor rank*, SIAM J. Matrix Anal. Appl. **30**(3) (2008), 1254–1279.

[8] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, *Scientific machine learning through physics-informed neural networks: where we are and what's next*, J. Sci. Comput. **92** (2022), 88.

[9] F. Dangel, T. Siebert, M. Zeinhofer, A. Walther, *Collapsing Taylor mode automatic differentiation*, Advances in Neural Information Processing Systems **38** (2025), 163307–163346.

[10] W. E, B. Yu, *The Deep Ritz method: a deep learning-based numerical algorithm for solving variational problems*, Commun. Math. Stat. **6**(1) (2018), 1–12.

[11] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, *DeepXDE: a deep learning library for solving differential equations*, SIAM Rev. **63**(1) (2021), 208–228.

[12] J. Sirignano, K. Spiliopoulos, *DGM: a deep learning algorithm for solving partial differential equations*, J. Comput. Phys. **375** (2018), 1339–1364.

[13] W. E, J. Han, A. Jentzen, *Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations*, Commun. Math. Stat. **5**(4) (2017), 349–380.

[14] A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM (2008).

[15] Z. Hao, J. Yao, C. Su, H. Su, Z. Wang, F. Lu, Z. Xia, Y. Zhang, S. Liu, L. Lu, J. Zhu, *PINNacle: a comprehensive benchmark of physics-informed neural networks for solving PDEs*, Advances in Neural Information Processing Systems **37**, Datasets and Benchmarks Track (2024).

[16] X. Huang, T. Alkhalifah, *Efficient physics-informed neural networks using hash encoding*, J. Comput. Phys. **501** (2024), 112760.

[17] A. Iarrobino, V. Kanev, *Power Sums, Gorenstein Algebras, and Determinantal Loci*, Lecture Notes in Mathematics 1721, Springer (1999).

[18] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, *Physics-informed machine learning*, Nat. Rev. Phys. **3** (2021), 422–440.

[19] E. Kharazmi, Z. Zhang, G. E. Karniadakis, *hp-VPINNs: variational physics-informed neural networks with domain decomposition*, Comput. Methods Appl. Mech. Engrg. **374** (2021), 113547.

[20] D. P. Kingma, J. Ba, *Adam: a method for stochastic optimization*, International Conference on Learning Representations (2015).

[21] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M. W. Mahoney, *Characterizing possible failure modes in physics-informed neural networks*, Advances in Neural Information Processing Systems **34** (2021).

[22] J. M. Landsberg, *Tensors: Geometry and Applications*, Graduate Studies in Mathematics 128, AMS (2012).

[23] K. Leng, M. Shankar, J. Thiyagalingam, *Zero coordinate shift: whetted automatic differentiation for physics-informed operator learning*, J. Comput. Phys. **505** (2024), 112904.

[24] R. Li, H. Ye, D. Jiang, X. Wen, C. Wang, Z. Li, X. Li, D. He, J. Chen, W. Ren, L. Wang, *A computational framework for neural network-based variational Monte Carlo with Forward Laplacian*, Nat. Mach. Intell. **6** (2024), 209–219.

[25] I. Loshchilov, F. Hutter, *Decoupled weight decay regularization*, International Conference on Learning Representations (2019).

[26] L. Lyu, Z. Zhang, M. Chen, J. Chen, *MIM: a deep mixed residual method for solving high-order partial differential equations*, J. Comput. Phys. **452** (2022), 110930.

[27] A. Paszke et al., *PyTorch: an imperative style, high-performance deep learning library*, Advances in Neural Information Processing Systems **32** (2019), 8024–8035.

[28] B. A. Pearlmutter, *Fast exact multiplication by the Hessian*, Neural Comput. **6**(1) (1994), 147–160.

[29] M. Raissi, P. Perdikaris, G. E. Karniadakis, *Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys. **378** (2019), 686–707.

[30] Z. Shi, Z. Hu, M. Lin, K. Kawaguchi, *Stochastic Taylor derivative estimator: efficient amortization for arbitrary differential operators*, Advances in Neural Information Processing Systems **37** (2024), 122316–122353.

[31] S. Tan, *TaylorDiff.jl: fast higher-order automatic differentiation in Julia*, software, GitHub repository (2022).

[32] S. Wang, Y. Teng, P. Perdikaris, *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*, SIAM J. Sci. Comput. **43**(5) (2021), A3055–A3081.

[33] S. Wang, X. Yu, P. Perdikaris, *When and why PINNs fail to train: a neural tangent kernel perspective*, J. Comput. Phys. **449** (2022), 110768.

[34] C. Wu, M. Zhu, Q. Tan, Y. Kartha, L. Lu, *A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks*, Comput. Methods Appl. Mech. Engrg. **403** (2023), 115671.